



Professional Program in Enterprise Data Science & Agentic AI with 

Data Science & MLOps on AWS

Enterprise Architectures for S3 Data Lakes, Serverless Analytics, SageMaker Distributed ML & Bedrock GenAI

Session Summary

01 S3 Data Lake Architecture & Partitioning Strategy

Storage classes, prefix request throughput, Hive-style partitioning, and durability.

02 Serverless Analytics with AWS Glue & Athena

Data Catalog, crawler schema discovery, Snappy Parquet columnar layout, and cost optimization.

03 Scalable ML with SageMaker & Distributed XGBoost

Managed container lifecycle, objective function math derivation, and Spot training.

04 Production MLOps, CI/CD & Real-Time Monitoring

SageMaker Pipelines DAG, real-time vs serverless endpoints, and statistical drift detection.

05 Enterprise Generative AI with Amazon Bedrock & RAG

Foundation models, Titan dense embeddings, vector retrieval, and zero-retention privacy.

Professional Program in Enterprise Data Science & Agentic AI with AWS — Modular Session Architecture

TOPIC 01

S3 Data Lake Architecture

Storage tiers, prefix sharding, and point-in-time partitioning.

Learning Objectives: S3 Data Lake Design

By the end of this session, learners can

- Design S3 bucket topologies optimized for high-throughput distributed parallel access.
- Eliminate prefix request throttling by structuring deterministic date/entity partition keys.
- Configure intelligent lifecycle policies that transition raw datasets to archive tiers.
- Enforce point-in-time correctness to prevent feature leakage across ML training pipelines.

S3 Cloud Storage Throughput & Durability

3,500

PUT / COPY / POST

Per partitioned prefix/sec

5,500

GET / HEAD

Requests/sec per prefix

11 9s

Data Durability

99.999999999% cross-AZ

Horizontal Prefix Partitioning

S3 automatically scales storage partitions behind distinct prefix paths. By prefixing telemetry logs with `s3://lake/events/year=2026/month=09/day=14/`, distributed readers achieve linear scaling across thousands of parallel workers.

Storage Tiering & Cost Optimization Matrix

S3 Standard & Express One Zone

- Single-digit millisecond data access.
- Express One Zone: Sub-10ms latency for real-time model feature retrieval.
- High-frequency read/write ML scratch space.

S3 Intelligent-Tiering & Glacier

- Automatically moves data based on access patterns.
- Zero operational overhead or retrieval fees.
- Glacier Flexible / Deep Archive: \$0.00099/GB/mo for compliance audit trails.

Caution – Avoid Sequential Monolithic Keys

Prefixes beginning with sequential auto-incrementing IDs (e.g. `id_0001`, `id_0002`) hash to identical S3 internal physical partitions, causing HTTP 503 SlowDown throttling under peak distributed reads.

Enterprise Data Lake Multi-Zone Topology

Data Zone	Format & Codec	Target AWS Service		Access Policy & SLA
Raw / Bronze	Raw JSON, CSV, GZIP	AWS Kinesis Firehose		Immutable landing zone; 30-day lifecycle to Glacier.
Clean / Silver	Snappy Parquet	AWS Glue ETL, EMR		Schema validated, deduplicated, partitioned by date.
Curated / Gold	Optimized Parquet / Delta	Amazon Athena	Redshift,	Aggregate feature tables; indexed for ML training.
Feature Store	Online (DynamoDB) / S3	SageMaker Store	Feature	Sub-10ms online inference lookup + offline historical join.

*Note: Immutable partitioned prefixes guarantee zero data corruption across multi-tenant reader clusters.

Point-in-Time Correctness & Anti-Leakage Rules

Point-in-Time Correctness

Every training feature value must be evaluated exclusively against information existing on or before the observation timestamp T_0 .

Caution – Temporal Leakage

Target

Using features updated after the business decision timestamp T_0 inflates offline validation metrics while causing catastrophic live degradation.

Important

Prefix partitioning with Hive standard YYYY/MM/DD enables partition pruning and linear IOPS scaling.

TOPIC 02

Serverless Engineering

Data

Cataloging with Glue, querying with Athena, and Parquet columnar layout.

AWS Glue Data Catalog & Crawlers

AWS Glue Data Catalog

METADATA

Unified Apache Hive-compliant metadata repository storing table schemas, partition statistics, and data types.

Tier: [Serverless Catalog](#)

Amazon Athena

ANALYTICS

Presto-powered distributed query engine reading structured data directly from S3 without cluster management.

Tier: [Serverless SQL](#)

Schema Evolution & Crawler Classification

Glue Crawlers inspect raw data files in S3, infer schemas (JSON, Parquet, CSV), detect schema drift, and update the Central Catalog automatically without manual DDL maintenance.

Columnar Format Efficiency: Parquet vs CSV

87%

Storage Savings

Snappy Parquet vs raw CSV

\$0.024

Per Query Cost

Scanned 4.8GB instead of 1.2TB

6.4×

Query Speedup

Vectorized columnar scan

Definition – Predicate Pushdown & Column Pruning

Because Apache Parquet stores data column-by-column with dictionary encoding and min/max row group metadata, Amazon Athena reads only the queried columns and skips irrelevant data blocks entirely.

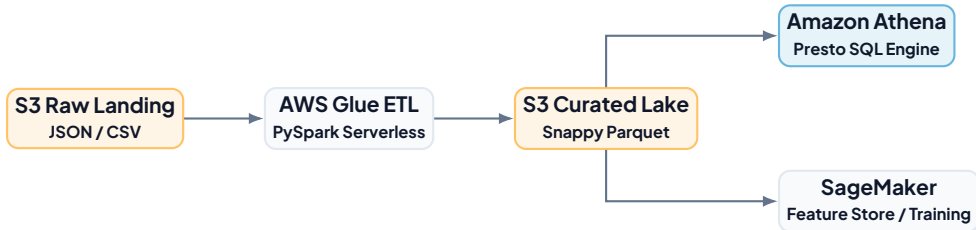
PySpark Glue ETL Pipeline

```
1 import sys
2 from awsglue.context import GlueContext
3 from pyspark.context import SparkContext
4
5 # Initialize Serverless Spark Execution Context
6 sc = SparkContext.getOrCreate()
7 glueContext = GlueContext(sc)
8
9 # DynamicFrame read from Catalog, partition and write Snappy Parquet
10 dyf = glueContext.create_dynamic_frame.from_catalog(database="telecom_db", table_name="raw_events")
11 glueContext.write_dynamic_frame.from_options(
12     frame=dyf, connection_type="s3",
13     connection_options={"path": "s3://lake-gold/events/", "partitionKeys": ["event_year", "event_month"]},
14     format="parquet", format_options={"compression": "snappy"}
15 )
16
```

Serverless Analytics Architecture

Serverless S3 to Athena & SageMaker Analytics Pipeline

PIPELINE



Zero server management; pay strictly per gigabyte scanned (\$5.00/TB scanned in Amazon Athena).

TOPIC 03

SageMaker Distributed Training

Managed Docker containers, objective function derivation, and Spot instances.

SageMaker Managed Training Architecture

Containerized Training Jobs

- Ephemeral compute clusters provisioned on demand.
- Automatic bootstrap: copies code, attaches EBS volumes, downloads dataset.
- Built-in algorithms (XGBoost, Linear Learner) or custom PyTorch/TensorFlow containers.
- Automatic termination immediately upon job completion.

Data Ingestion Modes

- **File Mode:** Downloads complete S3 dataset to instance EBS before training starts.
- **Fast File Mode:** Mounts S3 as a read-only POSIX filesystem without download wait.
- **Pipe Mode:** Streams dataset directly from S3 into FIFO pipes in memory.

Mathematical Derivation: XGBoost Objective & Taylor Expansion

STEP 1

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

Objective at boosting iteration t with tree complexity penalty

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

STEP 2

$$\tilde{\mathcal{L}}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Second-order Taylor expansion
where $g_i = \partial_{\hat{y}^{(t-1)}} l$ and $h_i = \partial_{\hat{y}^{(t-1)}}^2 l$

Mathematical Derivation: Optimal Leaf Weights & Split Gain

STEP 3

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}, \quad \mathcal{L}^* = -\frac{1}{2} \sum_{j=1}^T \frac{\left(\sum_{i \in I_j} g_i\right)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$$

Exact closed-form optimal leaf weights w_j^* for leaf partition I_j and optimal objective score $\mathcal{L}^*$

Structural Split Criterion

For a candidate tree split into left and right instance subsets I_L and I_R , the structural score gain is $\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$.

SageMaker Python SDK: Training with Spot Instances

```
1 from sagemaker.estimator import Estimator
2 import sagemaker
3
4 role = sagemaker.get_execution_role()
5 image_uri = sagemaker.image_uris.retrieve("xgboost", "us-east-1", "1.7-1")
6
7 xgb = Estimator(
8     image_uri=image_uri, role=role,
9     instance_count=2, instance_type="ml.m5.2xlarge",
10    use_spot_instances=True, max_run=3600, max_wait=7200,
11    checkpoint_s3_uri="s3://lake-ml-checkpoints/xgb-churn/"
12 )
13 xgb.set_hyperparameters(max_depth=5, eta=0.1, objective="binary:logistic", num_round=100)
14 xgb.fit({"train": "s3://lake-gold/train/", "validation": "s3://lake-gold/validation/"})
15
```

Training Optimization & Throughput Metrics

68%

Cost Savings

SageMaker Managed Spot training

14.2 GB/s

Pipe Mode Throughput

Zero local disk bottleneck

0.941

Validation ROC-AUC

Optimized tree ensemble

Important

Always configure checkpointing when using Spot instances to preserve state across node preemptions.

TOPIC 04

MLOps & Endpoint Lifecycle

SageMaker Pipelines, real-time endpoints, and statistical drift detection.

Production Inference Options Comparison

Real-Time Endpoints

- Dedicated multi-AZ EC2 instances (`ml.c6i.xlarge`).
- Sub-15ms p99 latency SLA.
- Auto-scales dynamically on request volume.
- Ideal for high-throughput live applications.

Serverless Inference

- Zero infrastructure provisioning or idle costs.
- Scales automatically from 0 to 200 concurrent workers.
- Memory configured from 1024MB to 6144MB.
- Ideal for intermittent, unpredictable burst workloads.

Shadow Deployments & Canary Routing

SageMaker allows routing 10% of live traffic to a new challenger model while mirroring 100% of requests to evaluate real-world performance without impacting production users.

SageMaker Model Monitor: Drift Detection

p99 11.4 ms

Inference Latency

Triton Server on g5.xlarge

Δ KS > 0.05

Drift Threshold

Kolmogorov-Smirnov alert

PSI > 0.20

Population Shift

Triggers automated retraining

Production Case Study – Automated Continuous Training Loop

When SageMaker Model Monitor detects feature attribution shift or Population Stability Index (PSI) > 0.2 on live scoring payloads stored in S3 Data Capture, an Amazon EventBridge rule triggers a SageMaker Pipeline execution to retrain on fresh data.

TOPIC 05

Enterprise Generative AI

Amazon Bedrock, Titan dense embeddings, vector retrieval, and RAG.

Data Science on AWS | Executive & Technical Masterclass

Amazon Bedrock Managed Foundation Models

Amazon Bedrock

FOUNDATION MODELS

Serverless API access to leading models: Claude 3.5 Sonnet, Amazon Titan, Llama 3, and Mistral Large.

Tier: [Serverless GenAI](#)

OpenSearch Serverless

VECTOR ENGINE

High-dimensional k-NN vector search engine powering Enterprise Knowledge Bases.

Tier: [Vector Database](#)

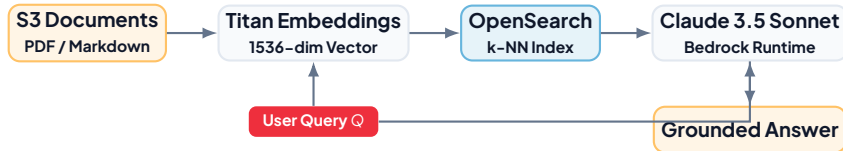
Enterprise Privacy Guarantee

Amazon Bedrock guarantees zero data retention: customer prompts and generated completions are never used to train foundation models, and all payloads remain encrypted under customer KMS keys.

Retrieval-Augmented Generation (RAG) Architecture

Enterprise Knowledge Base RAG Architecture

PIPELINE



Grounding LLMs on enterprise proprietary data eliminates hallucinations and provides auditable citation links.

Vector Similarity Mathematics in High Dimensions

STEP 1

$$\mathbf{e}_d = \text{Embed}_{\text{Titan}}(D), \quad \mathbf{e}_q = \text{Embed}_{\text{Titan}}(Q)$$

Dense representation vectors in $\mathbb{R}^{1536}$ with unit-norm normalization

STEP 2

$$\text{sim}(Q, D) = \cos(\theta) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2} = \sum_{k=1}^{1536} e_{q,k} \cdot e_{d,k}$$

Inner dot product for L_2 -normalized vector embeddings

STEP 3

$$\mathcal{N}_k(Q) = \arg \max_{D \in \mathcal{D}}^{(k)} [\text{sim}(Q, D)]$$

Hierarchical Navigable Small World (HNSW) approximate nearest neighbors

Amazon Bedrock Python SDK: Converse API

```
1 import boto3
2
3 bedrock = boto3.client("bedrock-runtime", region_name="us-east-1")
4
5 response = bedrock.converse(
6     modelId="anthropic.claude-3-5-sonnet-20240620-v1:0",
7     messages=[{"role": "user", "content": [{"text": "Analyze AWS Glue ETL latency bottlenecks."}]],
8     inferenceConfig={"maxTokens": 1024, "temperature": 0.2, "topP": 0.9},
9     system=[{"text": "You are a Principal AWS Solutions Architect specializing in Big Data & MLOps."}]
10 )
11 print(response["output"]["message"]["content"][0]["text"])
12
```

Enterprise GenAI Benchmark Metrics

200,000

Token Context

Claude 3.5 Sonnet window

140 ms

Time-to-First-Token

Optimized Titan inference

0%

Data Training Risk

KMS-encrypted zero retention

Important

Cloud Data Science is systems engineering: optimize data locality, automate pipelines, and monitor drift.